

database.py

SQLite schema and seed data for a social media app

```
import sqlite3
import os
import datetime

DB_PATH = os.path.join(os.path.dirname(__file__), "database.db")

def get_connection():
    conn = sqlite3.connect(DB_PATH)
    conn.row_factory = sqlite3.Row
    return conn

def init_db():
    conn = get_connection()
    cursor = conn.cursor()

    # Enable foreign keys
    cursor.execute("PRAGMA foreign_keys = ON;")

    # Table: Users
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS users (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            handle TEXT UNIQUE NOT NULL,
            name TEXT NOT NULL,
            bio TEXT,
            avatar TEXT,
            banner TEXT,
            joined_at TEXT NOT NULL
        )
    """)

    # Table: Posts
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS posts (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            user_id INTEGER NOT NULL,
            content TEXT NOT NULL,
            image_url TEXT,
            hashtags TEXT,
            created_at TEXT NOT NULL,
            FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
        )
    """)

    # Table: Likes
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS likes (
            user_id INTEGER NOT NULL,
            post_id INTEGER NOT NULL,
            created_at TEXT NOT NULL,
            PRIMARY KEY (user_id, post_id),
            FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
            FOREIGN KEY (post_id) REFERENCES posts(id) ON DELETE CASCADE
        )
    """)

    # Table: Reposts
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS reposts (
            user_id INTEGER NOT NULL,
            post_id INTEGER NOT NULL,
            created_at TEXT NOT NULL,
            PRIMARY KEY (user_id, post_id),
            FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
            FOREIGN KEY (post_id) REFERENCES posts(id) ON DELETE CASCADE
        )
    """)
```

```

# Table: Bookmarks
cursor.execute("""
    CREATE TABLE IF NOT EXISTS bookmarks (
        user_id INTEGER NOT NULL,
        post_id INTEGER NOT NULL,
        created_at TEXT NOT NULL,
        PRIMARY KEY (user_id, post_id),
        FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
        FOREIGN KEY (post_id) REFERENCES posts(id) ON DELETE CASCADE
    )
""")

# Table: Comments
cursor.execute("""
    CREATE TABLE IF NOT EXISTS comments (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        post_id INTEGER NOT NULL,
        user_id INTEGER NOT NULL,
        content TEXT NOT NULL,
        created_at TEXT NOT NULL,
        FOREIGN KEY (post_id) REFERENCES posts(id) ON DELETE CASCADE,
        FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
    )
""")

# Table: Follows
cursor.execute("""
    CREATE TABLE IF NOT EXISTS follows (
        follower_id INTEGER NOT NULL,
        following_id INTEGER NOT NULL,
        created_at TEXT NOT NULL,
        PRIMARY KEY (follower_id, following_id),
        FOREIGN KEY (follower_id) REFERENCES users(id) ON DELETE CASCADE,
        FOREIGN KEY (following_id) REFERENCES users(id) ON DELETE CASCADE
    )
""")

# Table: Messages
cursor.execute("""
    CREATE TABLE IF NOT EXISTS messages (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        sender_id INTEGER NOT NULL,
        receiver_id INTEGER NOT NULL,
        content TEXT NOT NULL,
        created_at TEXT NOT NULL,
        FOREIGN KEY (sender_id) REFERENCES users(id) ON DELETE CASCADE,
        FOREIGN KEY (receiver_id) REFERENCES users(id) ON DELETE CASCADE
    )
""")

# Table: Notifications
cursor.execute("""
    CREATE TABLE IF NOT EXISTS notifications (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        user_id INTEGER NOT NULL,
        actor_id INTEGER NOT NULL,
        type TEXT NOT NULL,
        target_id INTEGER,
        is_read INTEGER DEFAULT 0,
        created_at TEXT NOT NULL,
        FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
        FOREIGN KEY (actor_id) REFERENCES users(id) ON DELETE CASCADE
    )
""")

conn.commit()
conn.close()

seed_db()

def seed_db():
    conn = get_connection()
    cursor = conn.cursor()

```

```

# Check if users exist
cursor.execute("SELECT COUNT(*) as count FROM users")
if cursor.fetchone()["count"] > 0:
    conn.close()
    return

now = datetime.datetime.now().isoformat()
past_1h = (datetime.datetime.now() - datetime.timedelta(hours=1)).isoformat()
past_3h = (datetime.datetime.now() - datetime.timedelta(hours=3)).isoformat()
past_1d = (datetime.datetime.now() - datetime.timedelta(days=1)).isoformat()
past_2d = (datetime.datetime.now() - datetime.timedelta(days=2)).isoformat()

# Seed Users
users_data = [
    (1, "alex_dev", "Alex Rivera", "Full-Stack Engineer building modern web apps & AI tools. ■", "https://images.unsplash.com/photo-1506764425925-8973828a8494?ixlib=rb-1.2.1&auto=format&fit=crop&w=334&h=334&q=80", now),
    (2, "sarah_design", "Sarah Chen", "UI/UX Designer & Creative Director. Crafting minimalist digital experiences. ■", "https://images.unsplash.com/photo-1506764425925-8973828a8494?ixlib=rb-1.2.1&auto=format&fit=crop&w=334&h=334&q=80", now),
    (3, "tech_insider", "Tech Insider", "Daily news, breakdowns, and updates on AI, web development, and tech startup culture. ■", "https://images.unsplash.com/photo-1506764425925-8973828a8494?ixlib=rb-1.2.1&auto=format&fit=crop&w=334&h=334&q=80", now),
    (4, "creative_mind", "Elena Rostova", "Photographer & Digital Artist exploring light, color, and generative art. ■", "https://images.unsplash.com/photo-1506764425925-8973828a8494?ixlib=rb-1.2.1&auto=format&fit=crop&w=334&h=334&q=80", now),
]

cursor.executemany("""
    INSERT INTO users (id, handle, name, bio, avatar, banner, joined_at)
    VALUES (?, ?, ?, ?, ?, ?, ?)
""", users_data)

# Seed Follows
follows_data = [
    (1, 2, past_2d), # Alex follows Sarah
    (1, 3, past_2d), # Alex follows Tech Insider
    (2, 1, past_2d), # Sarah follows Alex
    (2, 4, past_2d), # Sarah follows Elena
    (3, 1, past_1d), # Tech Insider follows Alex
    (4, 2, past_1d), # Elena follows Sarah
]

cursor.executemany("INSERT INTO follows (follower_id, following_id, created_at) VALUES (?, ?, ?)", follows_data)

# Seed Posts
posts_data = [
    (1, 1, "Just launched our new real-time dashboard UI! Built using pure HTML5, modern CSS grid layout, and zero heavy dependencies. Check it out on our blog.", past_1h, "https://images.unsplash.com/photo-1506764425925-8973828a8494?ixlib=rb-1.2.1&auto=format&fit=crop&w=334&h=334&q=80", "#dashboard #ui #html5 #css #webdev"),
    (2, 2, "Pro tip for UI designers: Contrast and whitespace do 80% of the heavy lifting in UI clarity. Don't crowd your interface.", past_3h, "https://images.unsplash.com/photo-1506764425925-8973828a8494?ixlib=rb-1.2.1&auto=format&fit=crop&w=334&h=334&q=80", "#uiprotip #design #contrast #whitespace"),
    (3, 3, "Artificial Intelligence agents are accelerating frontend development workflows faster than ever in 2026. What are your thoughts?", past_1d, "https://images.unsplash.com/photo-1506764425925-8973828a8494?ixlib=rb-1.2.1&auto=format&fit=crop&w=334&h=334&q=80", "#ai #frontend #development #workflows"),
    (4, 4, "Golden hour reflections in the city. Captured with a 35mm prime lens. Architecture and geometry never fail to inspire.", past_2d, "https://images.unsplash.com/photo-1506764425925-8973828a8494?ixlib=rb-1.2.1&auto=format&fit=crop&w=334&h=334&q=80", "#photography #goldenhour #city #architecture #geometry"),
]

cursor.executemany("""
    INSERT INTO posts (id, user_id, content, image_url, hashtags, created_at)
    VALUES (?, ?, ?, ?, ?, ?)
""", posts_data)

# Seed Likes & Reposts & Bookmarks
likes_data = [
    (2, 1, past_1h),
    (3, 1, past_1h),
    (4, 1, past_1h),
    (1, 2, past_3h),
    (3, 2, past_3h),
    (1, 3, past_1d),
    (2, 4, past_1d)
]

cursor.executemany("INSERT INTO likes (user_id, post_id, created_at) VALUES (?, ?, ?)", likes_data)

reposts_data = [
    (2, 1, past_1h),
    (1, 3, past_1d)
]

cursor.executemany("INSERT INTO reposts (user_id, post_id, created_at) VALUES (?, ?, ?)", reposts_data)

bookmarks_data = [
    (1, 2, past_1h),
    (1, 3, past_1d)
]

cursor.executemany("INSERT INTO bookmarks (user_id, post_id, created_at) VALUES (?, ?, ?)", bookmarks_data)

# Seed Comments

```

```

comments_data = [
    (1, 1, 2, "This looks crisp, Alex! Loving the clean contrast and responsiveness.", past_1h),
    (2, 1, 3, "Super impressive load times! Are you serving this via a Python micro-server?", past_1h),
    (3, 2, 1, "100% agree Sarah. Whitespace is severely underrated in layout design.", past_3h),
    (4, 3, 1, "We've been using AI agentic coding tools like Antigravity. Game changer!", past_1d)
]
cursor.executemany("""
    INSERT INTO comments (id, post_id, user_id, content, created_at)
    VALUES (?, ?, ?, ?, ?)
""", comments_data)

# Seed Messages
messages_data = [
    (1, 2, 1, "Hey Alex! Loved your latest post on real-time dashboards.", past_1d),
    (2, 1, 2, "Thanks Sarah! Built it in pure Python & JS. How's your latest design project going?", past_1d),
    (3, 2, 1, "Going great! Re-building our design system components now.", past_1d),
    (4, 3, 1, "Hey Alex, we'd love to feature your PulseSocial project on Tech Insider!", past_3h)
]
cursor.executemany("""
    INSERT INTO messages (id, sender_id, receiver_id, content, created_at)
    VALUES (?, ?, ?, ?, ?)
""", messages_data)

# Seed Notifications
notifications_data = [
    (1, 1, 2, "like", 1, 0, past_1h),
    (2, 1, 3, "comment", 1, 0, past_1h),
    (3, 1, 2, "repost", 1, 0, past_1h),
    (4, 1, 3, "follow", None, 0, past_1d)
]
cursor.executemany("""
    INSERT INTO notifications (id, user_id, actor_id, type, target_id, is_read, created_at)
    VALUES (?, ?, ?, ?, ?, ?, ?)
""", notifications_data)

conn.commit()
conn.close()

if __name__ == "__main__":
    init_db()
    print("Database initialized successfully.")

```